Physical Database Design

5DV187 — Modern Database Systems and their Applications

Umeå University
Department of Computing Science

These slides are mostly taken verbatim, or with minor changes, from those prepared by
Stephen Hegner of Umeå University

Data Independence — a Basic Consideration

Data Independence refers to the condition that the functionality of the external user interface(s) to the DBMS be independent of the internal storage representation of the data.

- One of the fundamental features of the relational model is that it exhibits such independence by design.
- Nevertheless, it is important to have some understanding of the internal storage model, because certain choices of approach to queries may affect performance substantially.

Types of Access

- There are many different types of access which a comprehensive DBMS must support:

Key-based: Retrieval of data based upon the values of specific keys suggests an indexed or hashed strategy.

Sequential processing: Retrieval of large amounts of data in some order suggests that the data themselves should be stored in some appropriate order.

Range queries: Retrieval of data for which certain parameters fall within a range of values suggests that the above two approaches need to be combined.

Records

Record: The basic entity of storage in a DBMS.

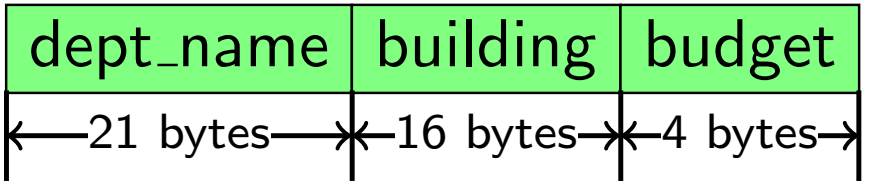
- In the ubiquitous *row-based* implementation of the relational model, each tuple is represented as a record.

Field: The basic physical data item.

- Each record is divided into one or more fields.
- In the usual implementation of the relational model, each field of a record corresponds to an attribute, with the field containing the value for that attribute.

Fixed-length record: The most common implementation is to allocate a fixed-size field for each attribute.

```
CREATE TABLE department
    (dept_name          VARCHAR(20),
     building           VARCHAR(15),
     budget             NUMERIC(12,2)
                  CHECK (budget > 0),
     PRIMARY KEY (dept_name)
    );
```



Physical Storage of Records

Blocks: Records are stored in units called *blocks*.

- A block usually corresponds to the sector size for the hard disk, or a small multiple of that size.

Blocking factor: The number of records which are stored in a block.

Unspanned blocking: Each record is contained entirely in one block.

Spanned blocking: A record may be split over (usually two) blocks.

- Rare in modern systems.

Organization of Records in Storage

- There are three fundamental ways in which records may be stored.

Heap: Any record may be stored anywhere.

- Typically, different record types are stored in distinct *files*.
- Access is entirely via indices and/or pointer chasing.

Ordered: The records are stored in the order defined by the value(s) of one more more attributes, typically but not always the primary key of the associated relation.

- Indices may still be used to facilitate both sequential and non-sequential access.

Hashed: The records are distributed into *buckets* according to some *hashing function*.

- Within each bucket, the records may be further organized according to one of the above two approaches.

Sequential Organization

Question: What does it mean for records on disk to be ordered?

- Here ordering on the ID (first) field of the `Instructor` relation is illustrated.

Question: But the records are stored in blocks. How are the blocks ordered?

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000

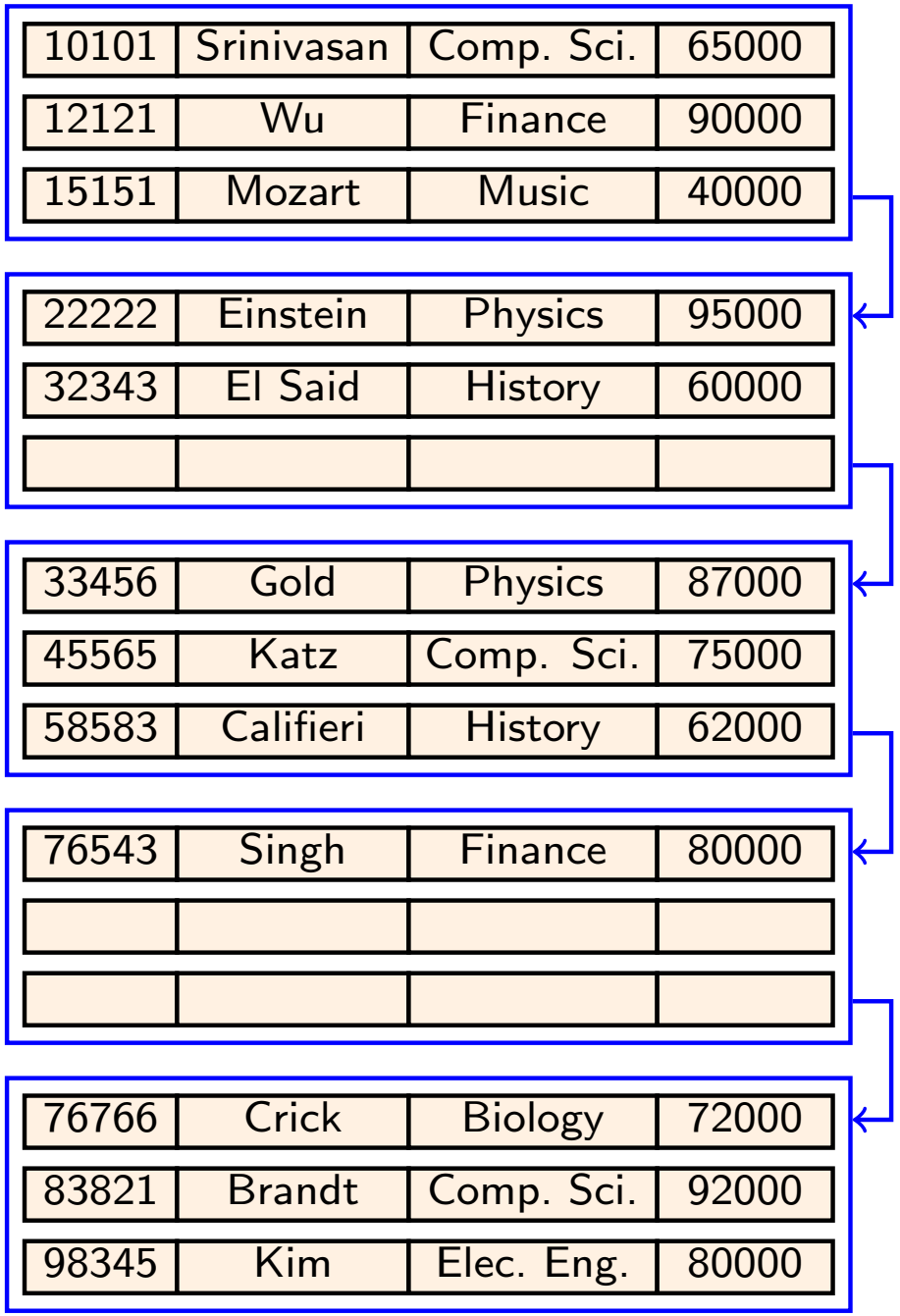
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000

45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000

76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Sequential Organization 2

- Information on the logical order of the blocks is maintained by the system,
- The system tries to keep blocks which are logical neighbors as physical neighbors as well.
- Within each block, the entries are ordered on the selected field.
- Blocks need not be full, but there may be a requirement on how "empty" they may be.



Classification of Indices

Index: An index is an access structure to records.

- The elements of the index are usually ordered for easy searching.

Classification: Indices may be classified along several dimensions.

Primary vs. secondary:

Primary (or clustering): Based upon the attribute(s) used to order the records.

- Need not be built on the primary key (but often is).

Secondary (or non-clustering): Not primary.

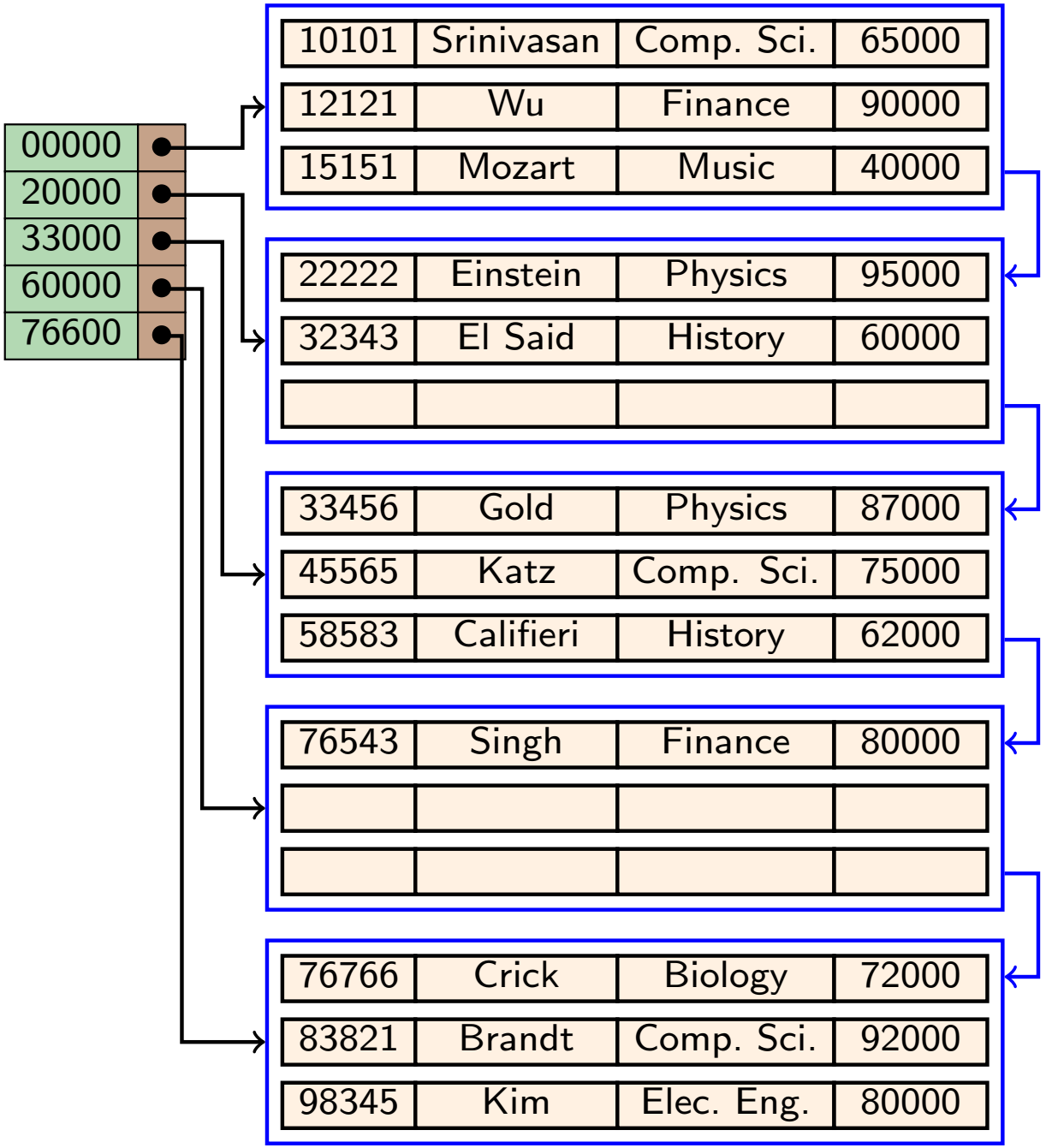
Dense vs. non-dense:

Dense: There is an index entry for each value of the search key which occurs in the relation.

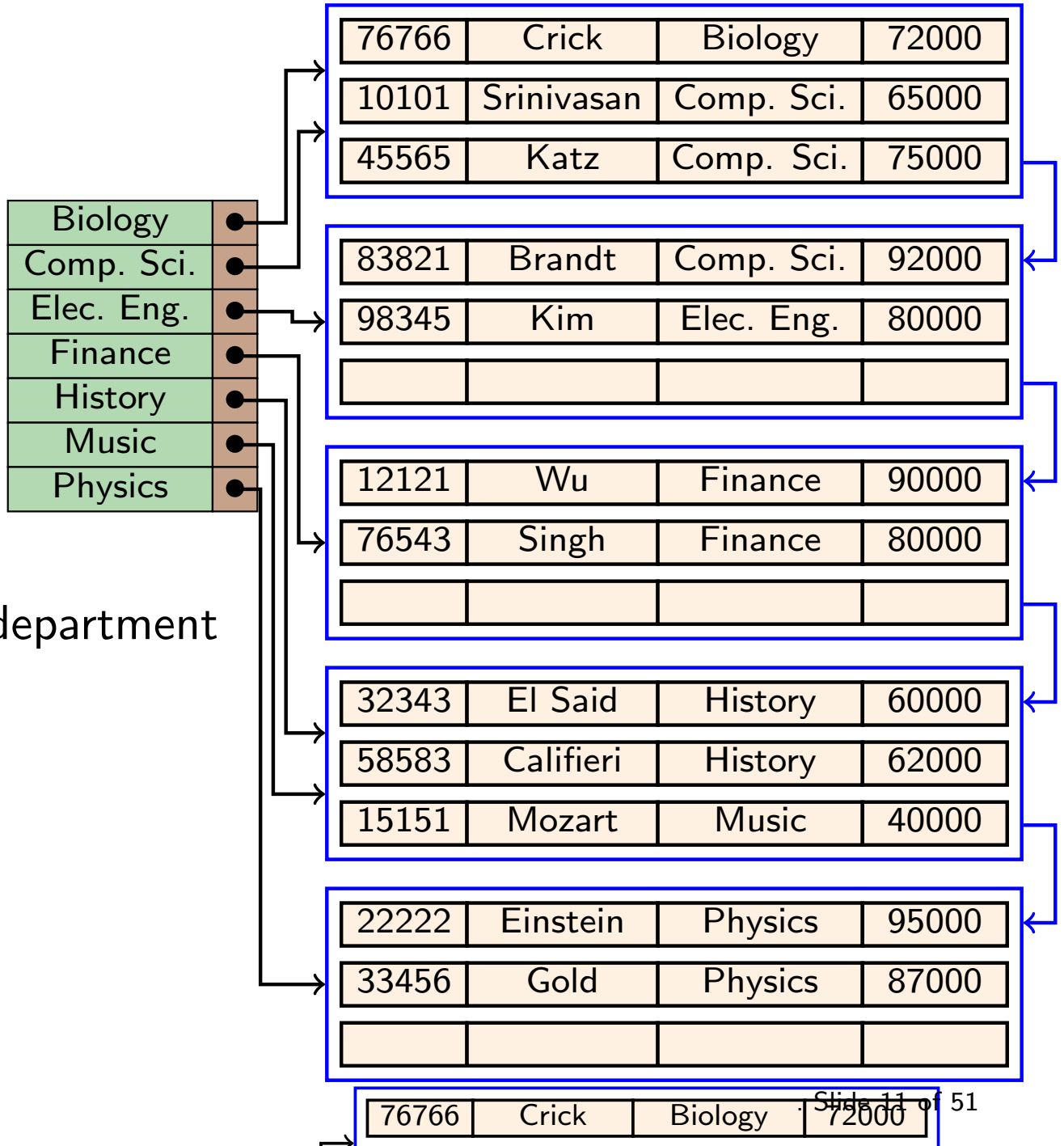
Non-dense (or sparse): Not dense.

A Sparse Clustering Index on the Primary Key

- The index values need not be key values of records which are currently in the database.
- Each link points to the first block containing an entry greater than or equal to the index value.
- Usually, with such a non-dense index, if an index link points to a block B, then all entries in B are greater than or equal to the index value.



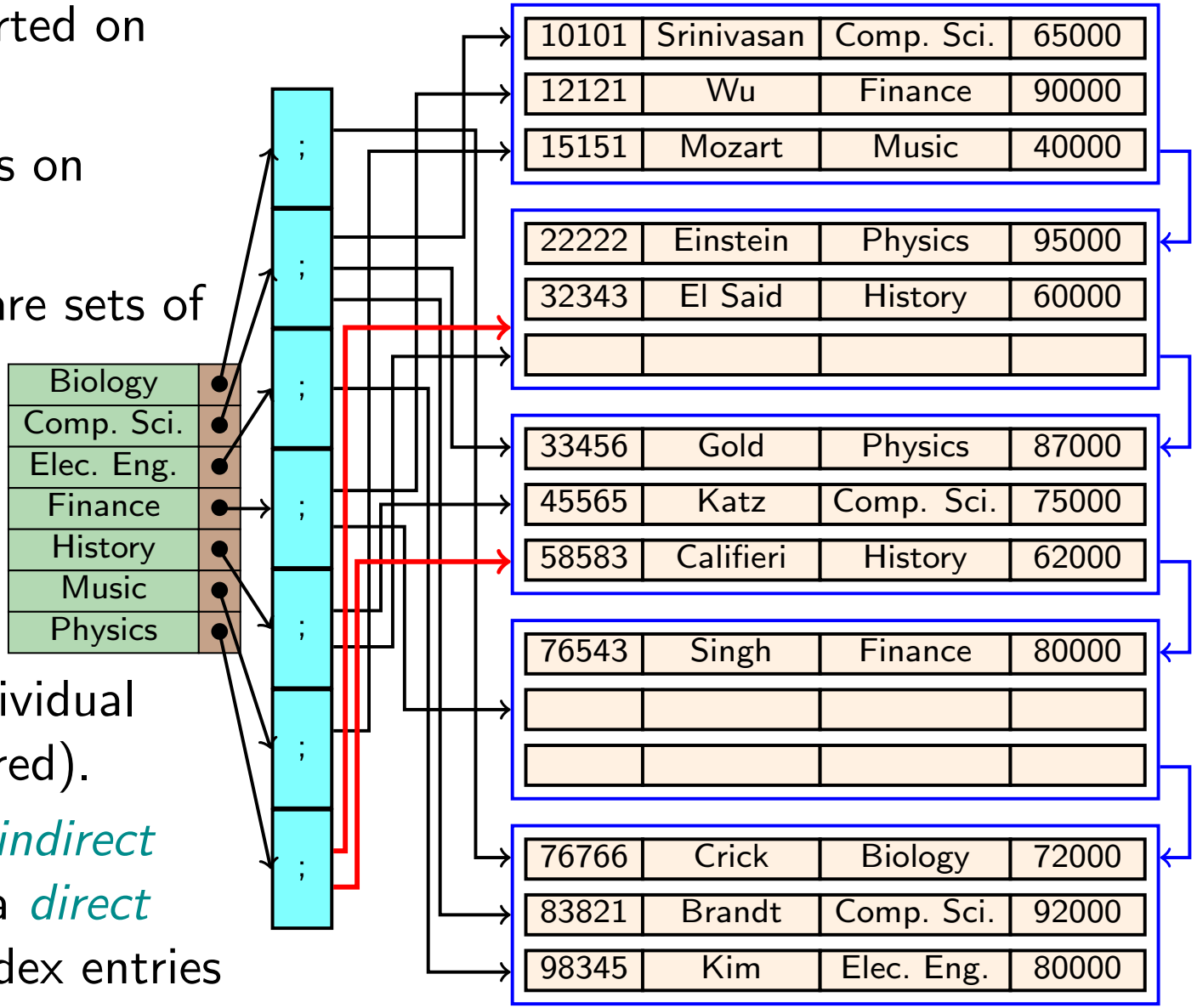
A Dense Clustering Index Not on the Primary Key



- The records are sorted by department name in this example.
- There is an index entry for every department name which occurs in the database, but not for every possible department name.

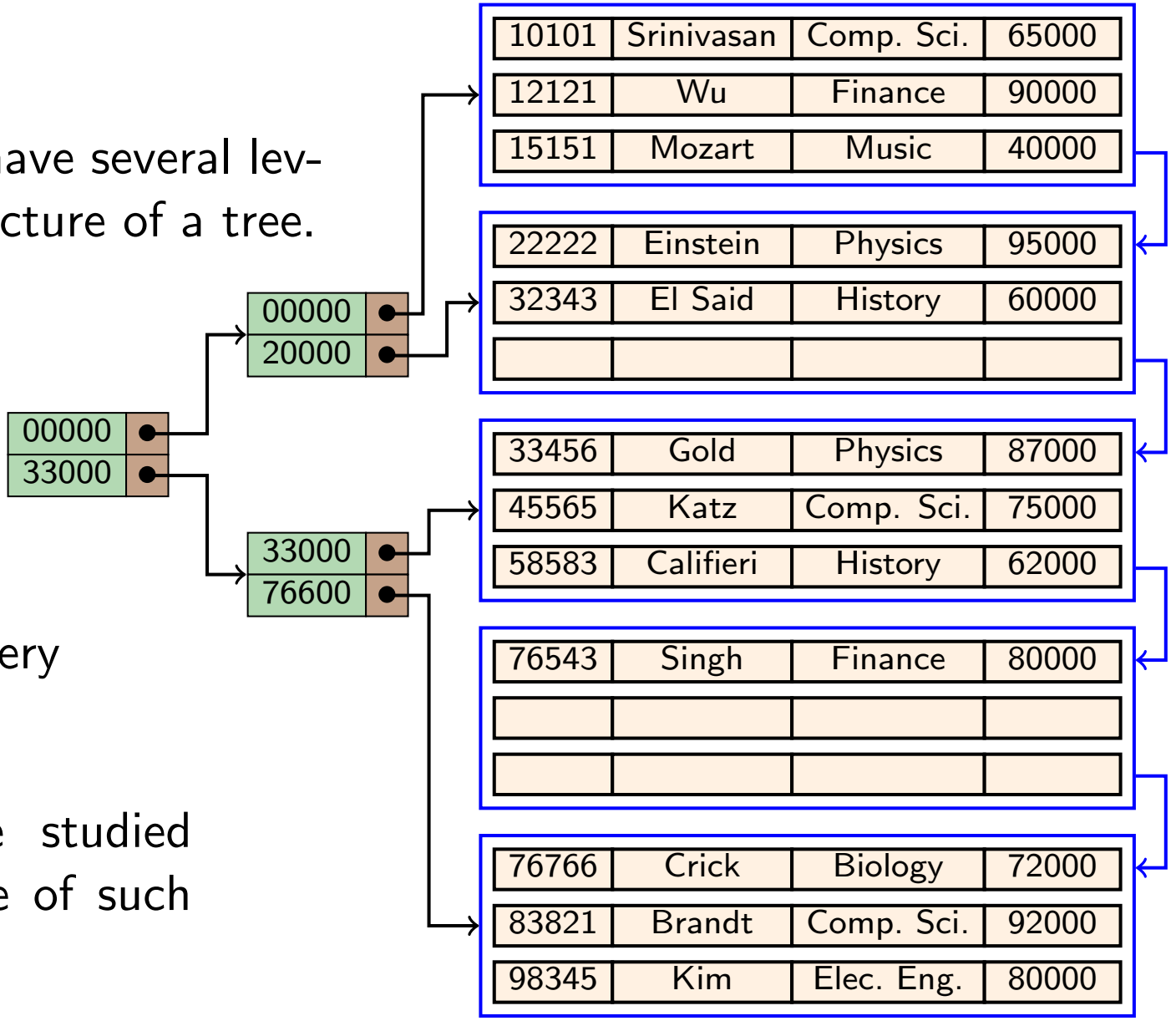
A Non-Clustering Index

- The example file is sorted on employee ID.
- The secondary index is on department.
- The blocks in aqua are sets of pointers for the given value of the index attribute.
- Note that a pointer from such a set leads to a block, not an individual record. (Examples in red).
- This is also called an *indirect* index, as opposed to a *direct* index, in which the index entries point directly to the record blocks.



A Multi-Level Index

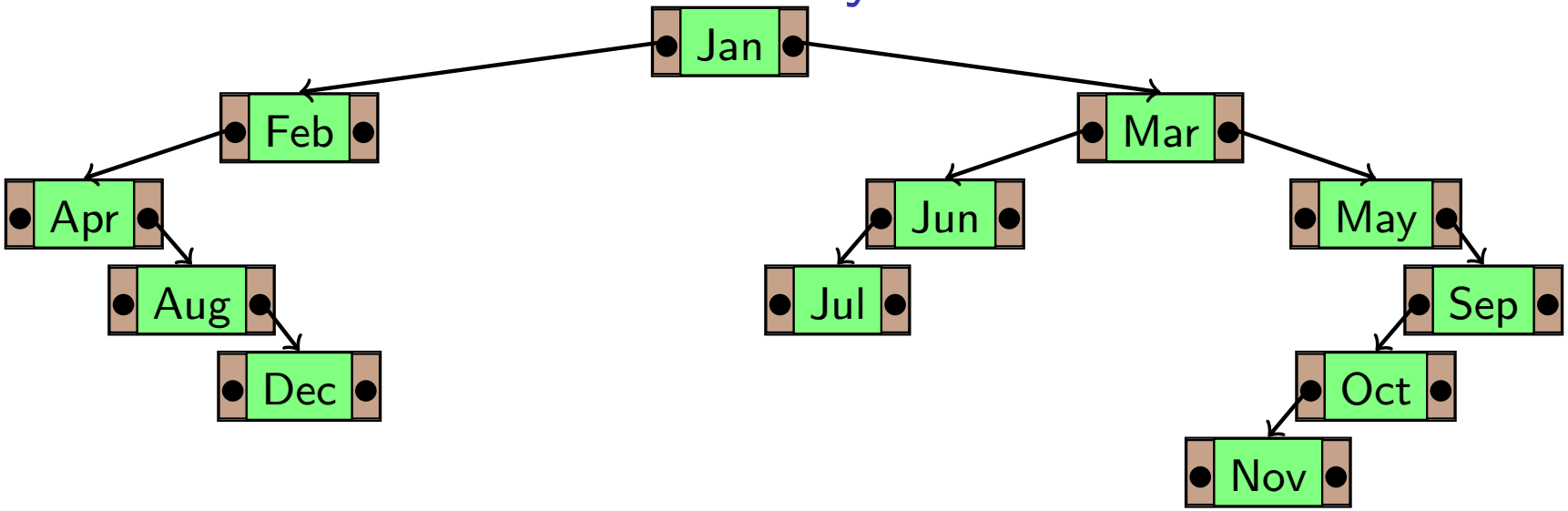
- The index itself may have several levels, usually in the structure of a tree.
- Illustrated here is a multi-level non-dense primary index on the instructor ID.
- Such indices are very common.
- The B^+-tree, to be studied shortly, is an example of such an index structure.



B^+-Trees

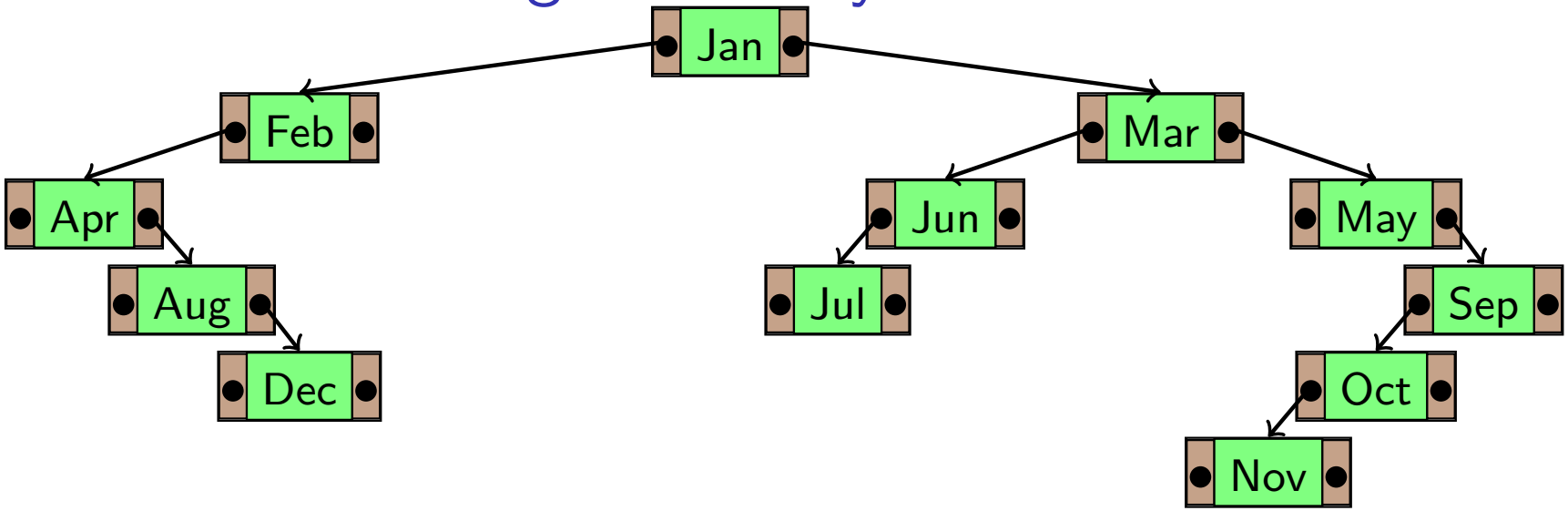
- The most important form of index structure in database systems is the *B^+-tree*.
- B+-trees are an extension of the classical and ubiquitous binary search tree (which everyone in this class should already know.)

Review of Binary Search Trees



- Shown above is the binary tree obtained by inserting, into an initially empty tree, the three-letter abbreviations for the months, in chronological order.
- The method of search is standard:
  - Begin at the root.
  - If the element is found, stop.
  - Otherwise, go left if the item sought is less than the value of the current vertex, else go right.
  - Repeat until found or an empty pointer is reached.

Shortcomings of Binary Trees for Database Storage



- Binary search trees have two shortcomings which renders them a poor choice for database storage.

No guaranteed balance: Binary search trees need not be balanced, and unless special measures are taken, can grow far out of balance.

- Lack of balance can lead to long searches, with even average case time $O(n)$ rather than $O(\log(n))$, $n =$ number of vertices.

Much pointer following: One pointer must be followed for each decision in the search process.

- In the DBMS context, following a pointer often involves a disk read, rendering the approach unusably slow.

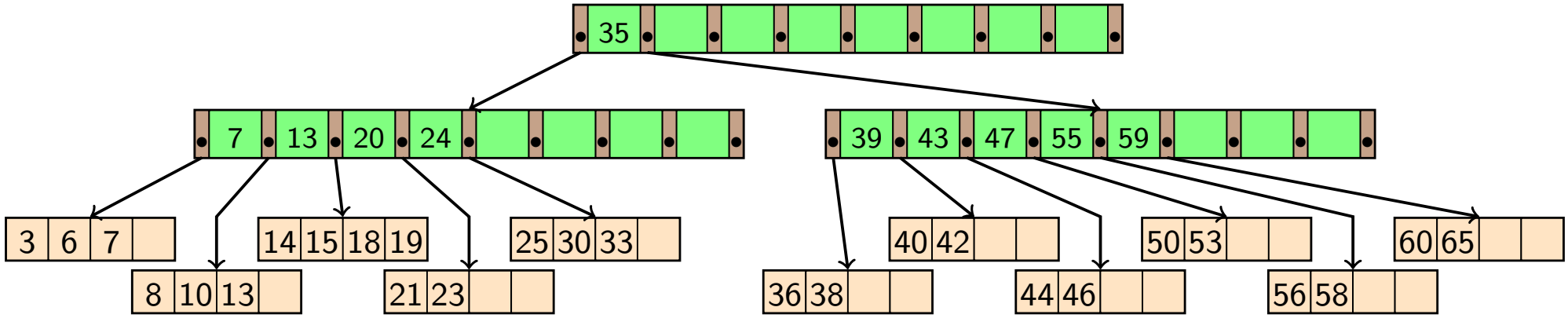
The B^+-Tree

- The B^+-tree
  - The tree is balanced (i.e. The length to each leave is the same for all leaves).
  - All records are stored in the leaves.
  - The internal vertices contain the index only.

Advantages:

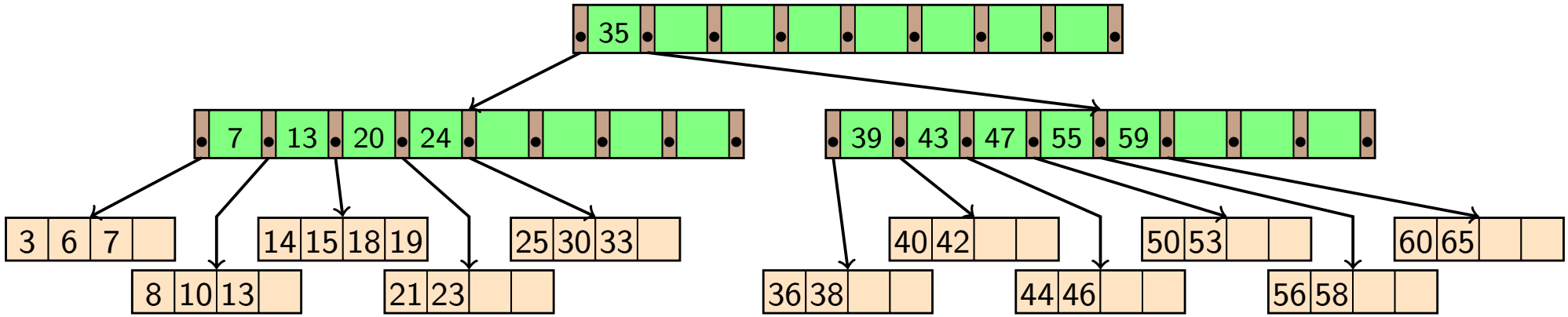
  - Since index fields are typically much smaller than record fields, many index values may be stored in a single internal vertex.
  - This implies that the fanout in the non-leaf vertices will be very high.
  - This implies, in turn, that the index will be relatively small and not very deep.
  - The leaf vertices (left to right) form an ordered sequential representation, thus facilitating sequential processing.

Visualization of a B^+-Tree



- Shown above is a B^+-tree of *order* (9,4).
  - The order of a non-leaf vertex is defined as maximum number of pointers.
  - The order of a leaf vertex is defined to be the maximum number of records which can be stored in it.
- Note that leaf vertices do not have any pointer fields (none are needed).
- The values which are stored in the non-leaf vertices are just *possible* keys, and do not need to be key values of records stored in the leaves.
- A key value does not occur more than once in the index.

Convention for Index Paths in a B^+-Tree



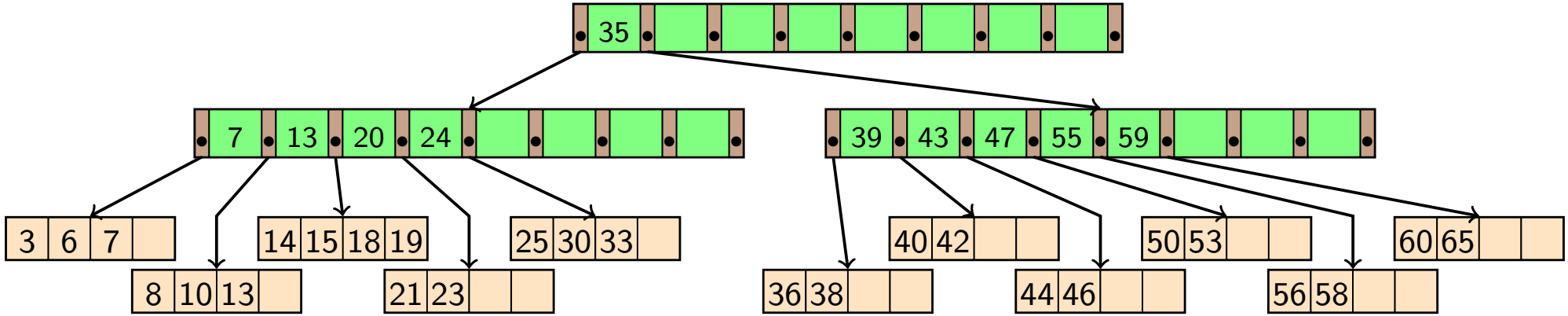
Convention for pointers of index vertices:

Pointer to the left of key k: All further indices and records with keys which are $\leq k$.

Pointer to the right of key k: All further indices and records with keys which are $> k$.

- In other words, for a search value which is equal to the index value, go left, not right.

Fullness Conditions on the Vertices of a B^+-Tree



- All vertices except the root must be at least "half full".

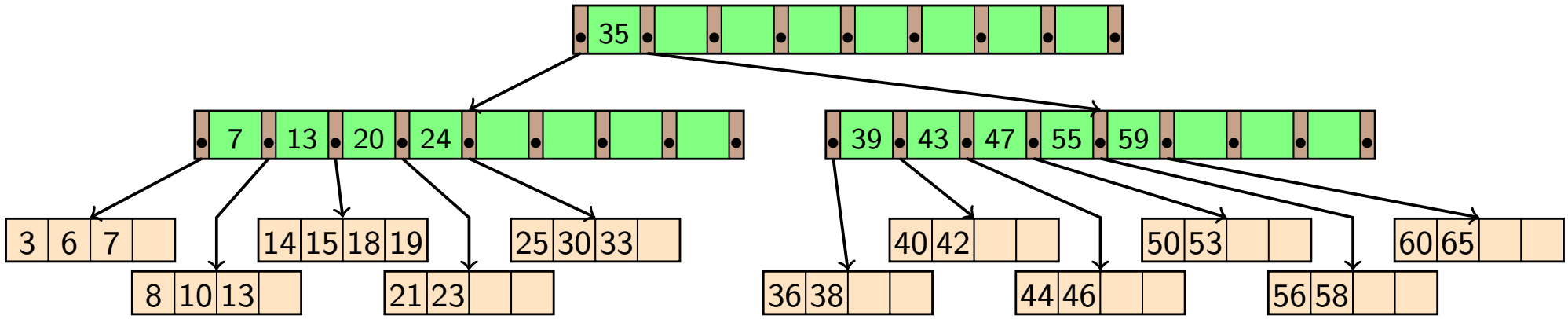
Internal (index) vertices: The condition for internal (index vertices) is:

- Each vertex except the root must contain at least $\lfloor (n_{\text{int}} - 1)/2 \rfloor$ vertices, where n_{int} is the order (number of pointers) in such a vertex.

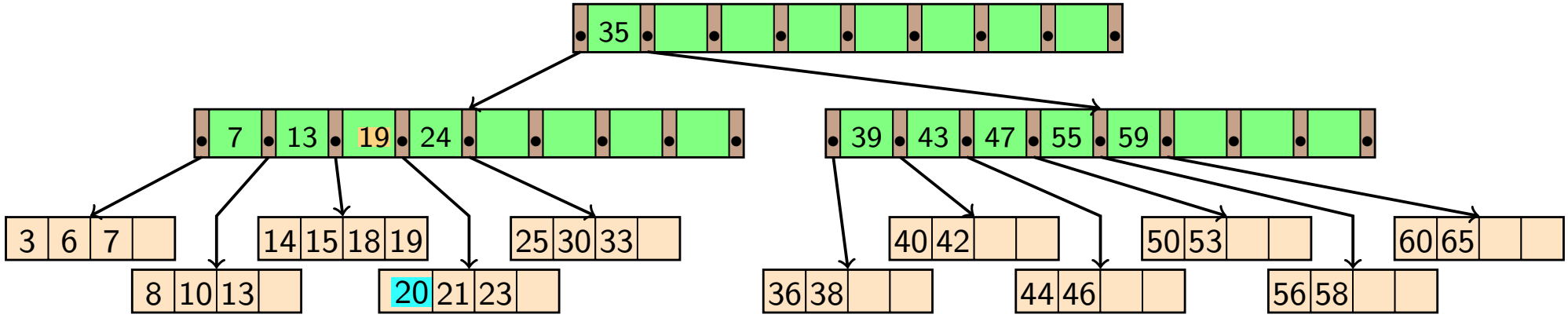
Leaf vertices: The condition for leaf vertices stipulates that if the maximum number of records is odd, then half full is defined by "round up".

- Each leaf must contain at least $\lceil (n_{\text{ext}})/2 \rceil$ vertices, where n_{ext} is the order (number of possible records) in such a vertex.

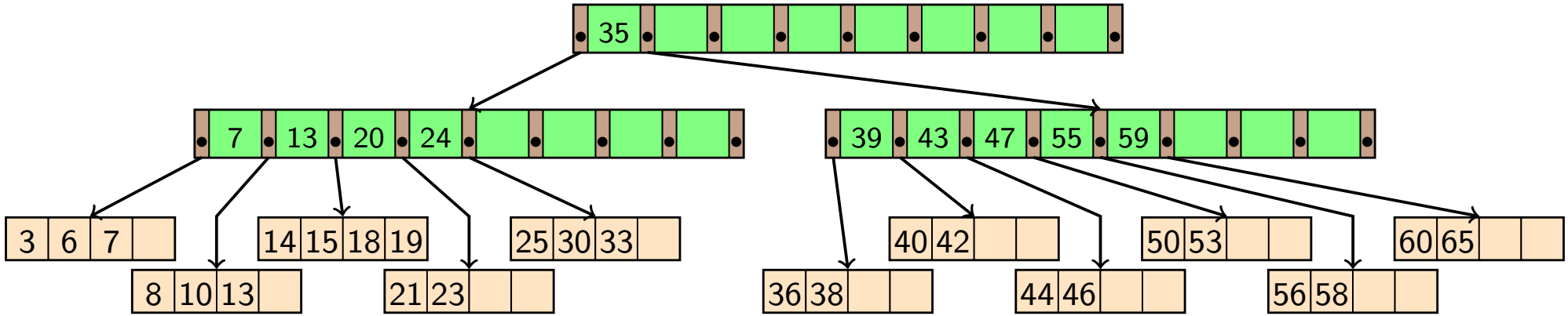
Insertion into a B^+-Tree



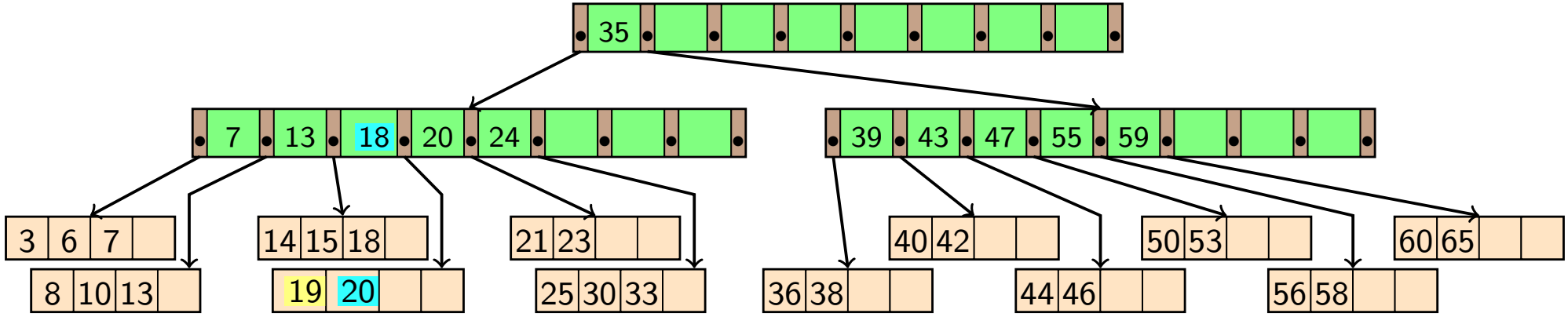
- Consider insertion of a record with key 20 into the above tree.
- The index value 20 must be changed to 19 (changes shown in orange).
- Alternatively, a straightforward rotation may be used.



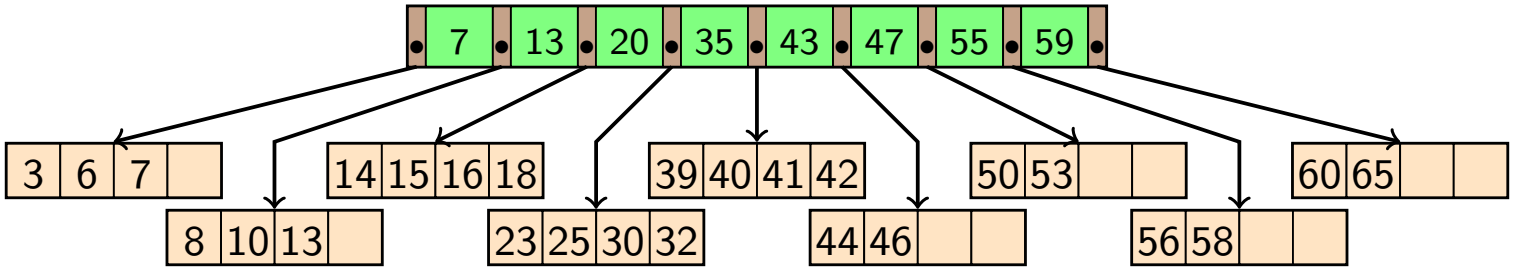
Insertion into a B^+-Tree — 2



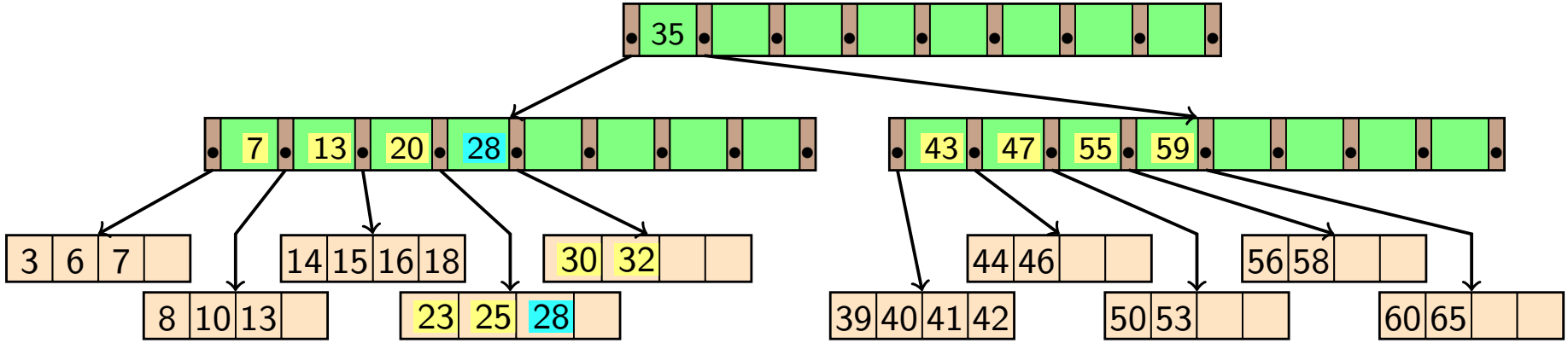
- It is possible to solve this same insertion of 20 via a split of the leaf vertex together with the insertion of a new index value.



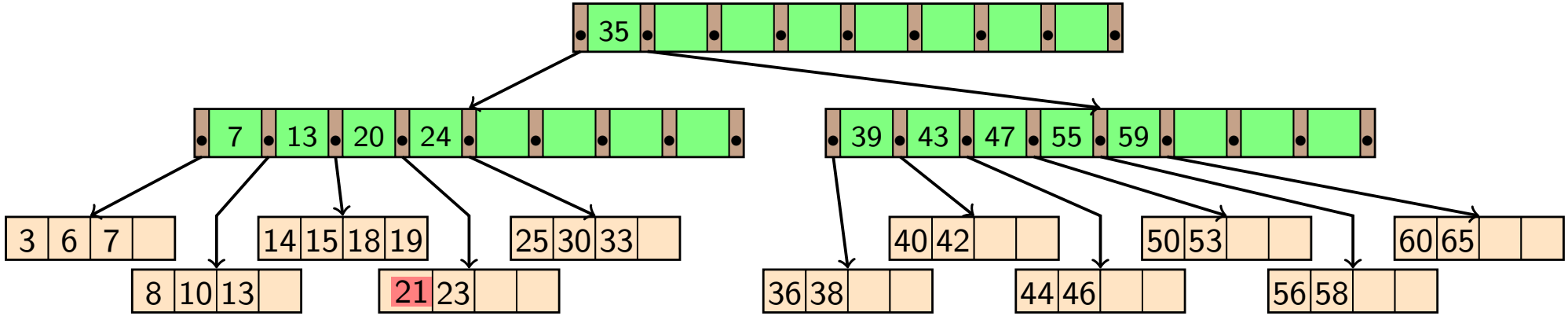
Insertion into a B^+-Tree — 3



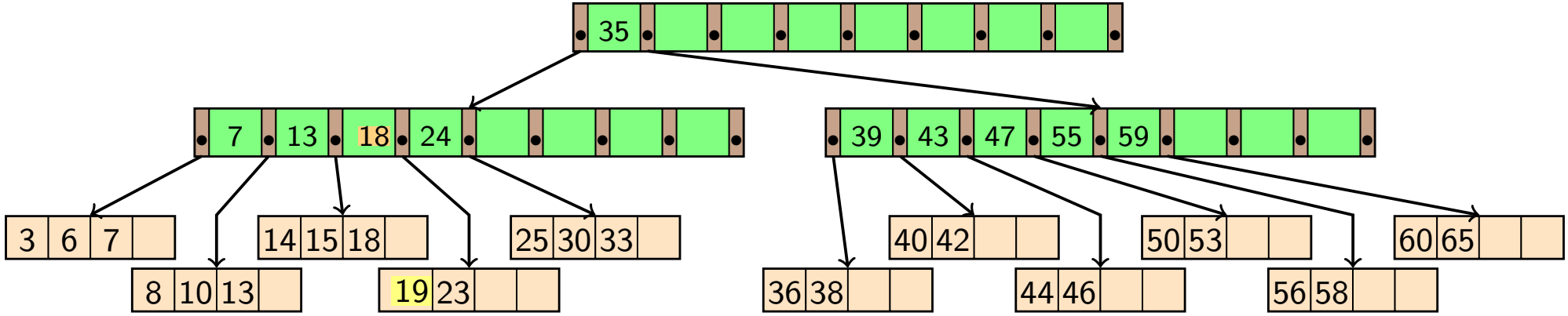
- Insertion of a record with key 28 into the above tree requires a split of the vertex at the second level as well as the root.
- The inserted internal key (not record) 28 could be either of 28 or 29.
- This is the only way which the depth of a B^+-tree may increase.



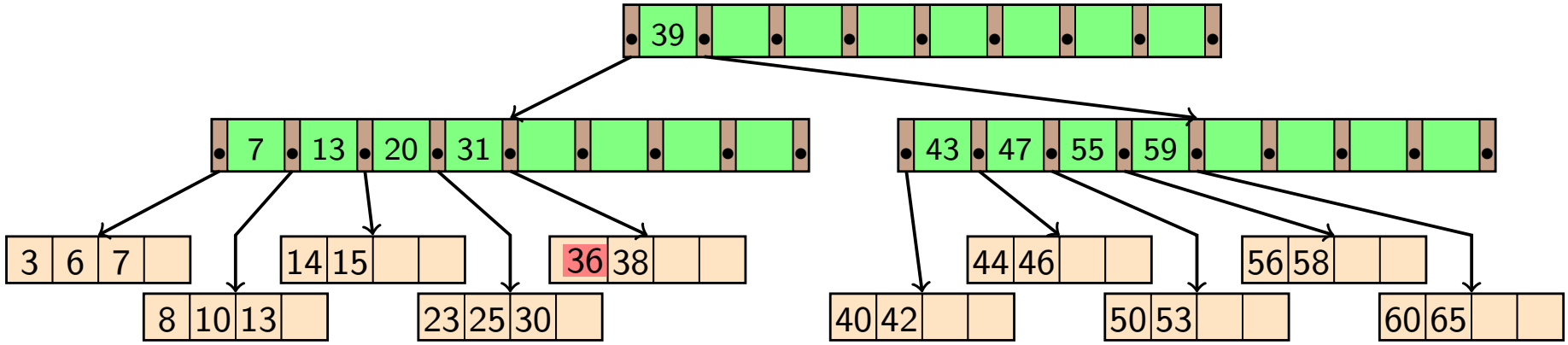
Deletion from a B^+-Tree



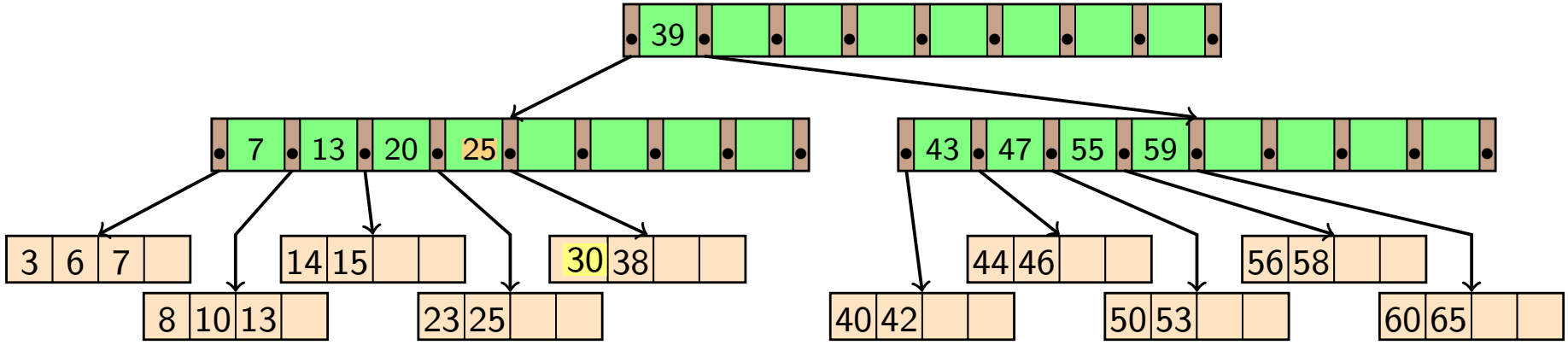
- Deletion of 21 from the above tree is realized as shown below.
- A simple rotation and change of key value is required.



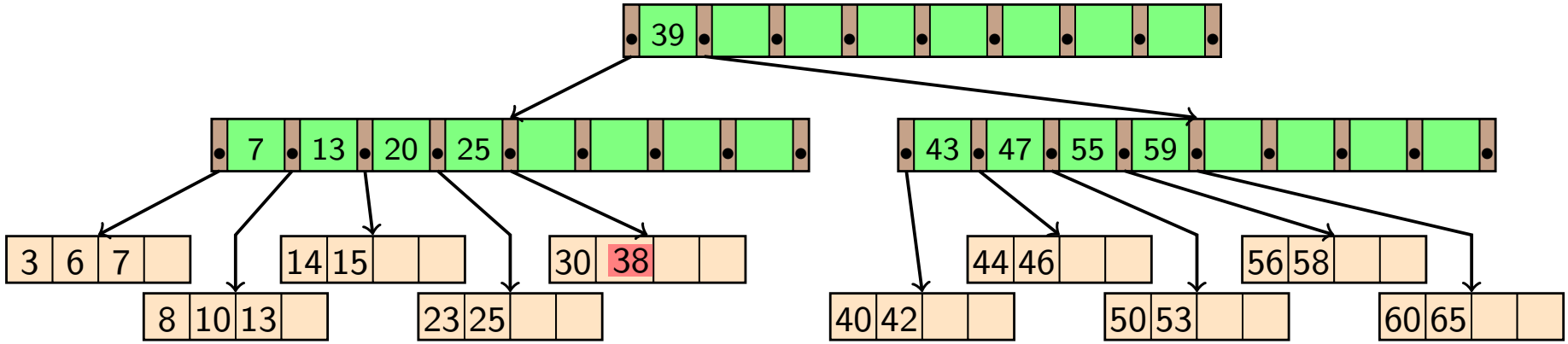
Deletion from a B^+-Tree — 2



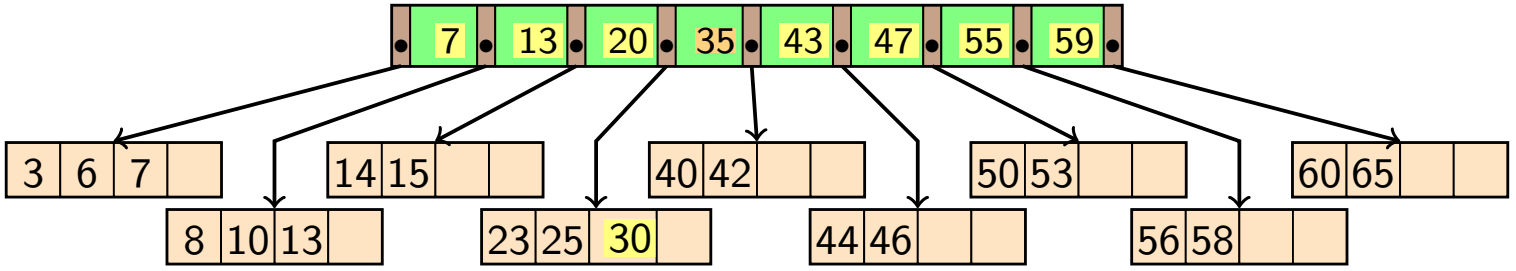
- Deletion of 36 from the above tree is realized as shown below.
- A simple rotation and change of key value is required.



Deletion from a B^+-Tree — 3

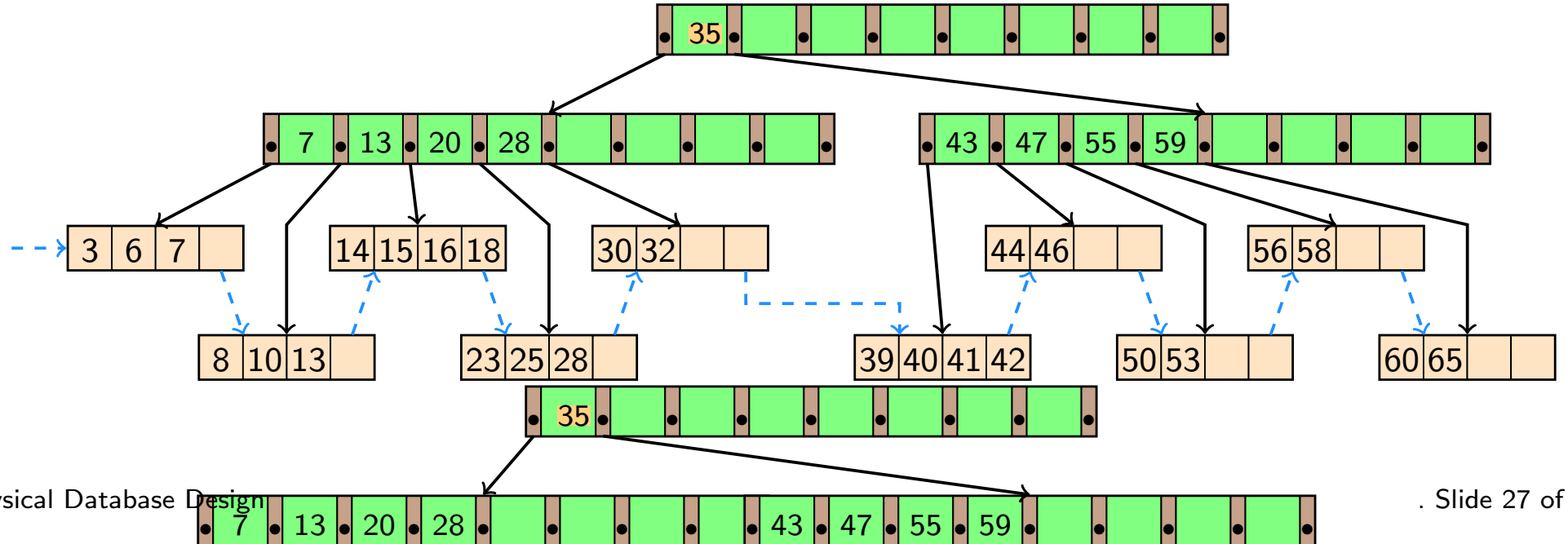


- Continuing with the previous result, deletion of 38 requires a combination of both vertices and keys, together with shrinking of the depth.
- The new value for the key obtained by combining 20 and 39 (35) could be any value 30-39.
- This is the only way which the depth of a B^+-tree may become smaller.



Sequential Access in B^+-Trees

- Sequential access may be obtained by linking the leaves together.
- Usually, links are provided in both directions, so that reverse as well as forward sequential access is possible.
- This also provides efficient access to neighboring data vertices.
- For best performance, adjacent leaf vertices should be sequential neighbors on the disk as well, insofar as possible.



Depth of B^+-Tree

Example setting:

Page size:	2 KBytes
Record size:	128 Bytes
Pointer size:	4 Bytes
Bytes per internal key	16
Total records	10^6
Total bytes for sequential pointers in leaves	8

- Maximum order n for the internal vertices:

$$(n \times \text{PtrSize}) + ((n-1) \times \text{KeySize}) \leq \text{PageSize}$$

$$n = \left\lfloor \frac{\text{PageSize} + \text{KeySize}}{\text{PtrSize} + \text{KeySize}} \right\rfloor = \left\lfloor \frac{2048 + 16}{4 + 16} \right\rfloor = 103$$

- Maximum number of records r_{max} per leaf vertex:

$$(r_{\text{max}} \times \text{RecSize}) + \text{SeqPtrsSize} \leq \text{PageSize}$$

$$r_{\text{max}} = \left\lfloor \frac{\text{PageSize} - \text{SeqPtrsSize}}{\text{RecSize}} \right\rfloor = \left\lfloor \frac{2048 - 8}{128} \right\rfloor = 15$$

Maximum-Depth B^+-Trees – Example Computation

Minimum density: A B^+-tree will have maximum depth when it has minimum *density* — as few keys per internal vertex and as few records per leaf as possible.

- Internal vertices other than the root will contain $\lfloor (n-1)/2 \rfloor = \lfloor 102/2 \rfloor = 51$ keys.
- The root will contain one key.
- Record vertices will contain $\lceil r_{\max}/2 \rceil = \lceil 15/2 \rceil = 8$ records.
- Brute force:

Level	Vertices at level	Keys at the level	Min Leaf Records
root	1	1	$2 \cdot 8 = 16$
1	2	$2 \times 51 = 102$	$2 \times 52 \times 8 = 832$
2	$2 \times 52 = 104$	$104 \times 51 = 5304$	$104 \times 52 \times 8 = 58240$
3	$104 \times 52 = 5408$	$5408 \times 51 = 275808$	$5408 \times 52 \times 8 = 2249728$

- The maximum depth of the index is thus 2, since a depth of 3 would require at least 2249728 records.

Parameters of B^+-Trees

- The brute-force approach becomes tedious, particularly when the depth becomes substantial.
- It is instructive to develop more general formulas.
- The general parameters are as follows:

Parameter	Meaning
m	number of keys in the root vertex
q	number of keys in other internal vertices
r	number of records in a leaf vertex
d	depth, from root to leaf

- It is very rare that all non-root vertices will contain exactly the same number of records.
- These parameters are therefore used in approximation.
- In the above example, $m = 1$, $q = 51$, $r = 8$, and d is to be computed.
- A B^+-tree which satisfies these conditions will be called *(m, q, r, d)-uniform*.

Maximum-Depth B$^+$-Trees — Formulas

- Here is a computation of the number of vertices at each level.

Level	Index Vertices	Keys	Total Rec Next Level
root	1	m	$(m+1)\cdot r$
1	$m+1$	$(m+1)\cdot q$	$(m+1)\cdot(q+1)\cdot r$
2	$(m+1)\cdot(q+1)$	$(m+1)\cdot(q+1)\cdot q$	$(m+1)\cdot(q+1)^2\cdot r$
3	$(m+1)\cdot(q+1)^2$	$(m+1)\cdot(q+1)^2\cdot q$	$(m+1)\cdot(q+1)^3\cdot r$
$\ldots$	$\ldots$	$\ldots$	$\ldots$
$d-1$	$(m+1)\cdot(q+1)^{d-2}$	$(m+1)\cdot(q+1)^{d-2}\cdot q$	$(m+1)\cdot(q+1)^{d-1}\cdot r$
d	$(m+1)\cdot(q+1)^{d-1}$	$(m+1)\cdot(q+1)^{d-1}\cdot q$	$(m+1)\cdot(q+1)^{d}\cdot r$

- The total number of records $R(m,q,r,d)$ in an (m,q,r,d)-uniform B$^+$-tree is given by choosing the value for level $d-1$ (the last level of indices) in the table:

$$R(m,q,r,d) = (m+1)\cdot(q+1)^{d-1}\cdot r$$

- Solving for d:

$$d = \log_{q+1}\left(\frac{R(m,q,r,d)}{(m+1)\cdot r}\right)+1 = \frac{\log_e\left(\frac{R(m,q,r,d)}{(m+1)\cdot r}\right)}{\log_e(q+1)}+1$$

Maximum-Depth B^+-Trees — the Formulas on the Example

- Continuing with:

$$d = \log_{q+1}\left(\frac{R(m,q,r,d)}{(m+1)\cdot r}\right) + 1 = \frac{\log_e\left(\frac{R(m,q,r,d)}{(m+1)\cdot r}\right)}{\log_e(q+1)} + 1$$

- In the example, $r = 8$, $N = 1000000$, $m = 1$ and $q = 51$, so

$$d = \frac{\log_e\left(\frac{1000000}{(1+1)\cdot 8}\right)}{\log_e(51+1)} + 1 = \frac{\log_e(62500)}{\log_e(52)} + 1 = 3.79$$

- Since the depth of a B^+-tree must be an integer, it follows that it cannot be greater than $\lfloor 3.79 \rfloor = 3$, in agreement with the brute-force approach.

Minimum-Depth B^+-Trees – Example Computation

Maximum density: A B^+-tree will have minimum depth when it has maximum *density* — as many keys per internal vertex and as many records per leaf as possible.

- Internal vertices, including the root, will contain $n - 1 = 102$ records.
- Record vertices will contain $r_{\max} = 15$ records.
- Brute force:

Level	Vertices at level	Keys at the level	Leaf Records
root	1	102	$103 \cdot 15 = 1545$
1	103	$103 \times 102 = 10506$	$103^2 \times 15 = 159135$
2	103^2	$103^2 \times 102 = 1082116$	$103^3 \times 15 = 16390905$

- The minimum depth of the index is thus 2, since a depth of 1 would support at most 159135 records.

Minimum-Depth B^+-Trees — Applying the Formula

- Recall:

$$d = \log_{q+1}\left(\frac{R(m,q,r,d)}{(m+1)\cdot r}\right) + 1 = \frac{\log_e\left(\frac{R(m,q,r,d)}{(m+1)\cdot r}\right)}{\log_e(q+1)} + 1$$

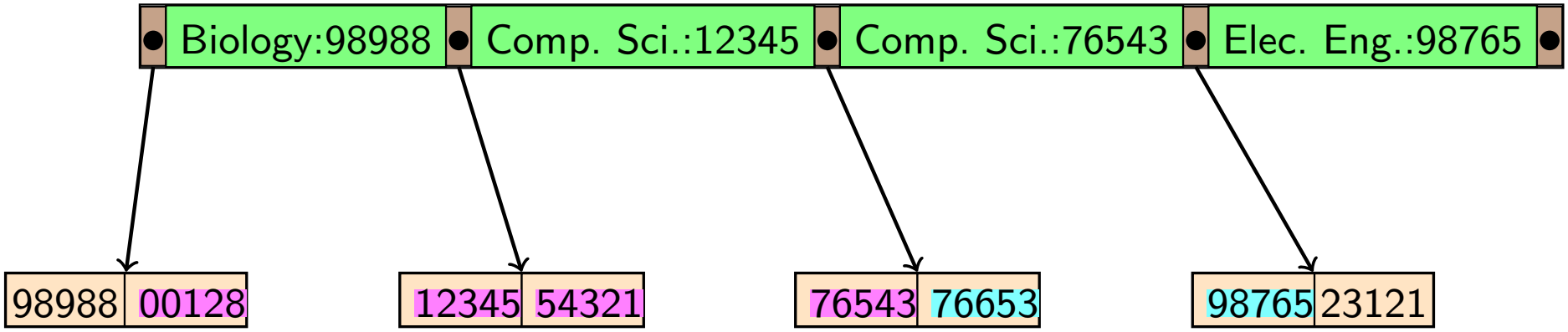
- In the example, $r = 15$, $N = 1000000$, $m = q = 102$, so

$$d = \frac{\log_e\left(\frac{1000000}{(102+1)\cdot 15}\right)}{\log_e(102+1)} + 1 = \frac{\log_e(647.24)}{\log_e(103)} + 1 = 2.39$$

- Since the depth of a B^+-tree must be an integer, it follows that it cannot be less than $\lceil 2.39 \rceil = 3$, in agreement with the brute-force approach.

Secondary Search Keys for B^+-Trees

- The records of a B^+-tree can only be ordered on one attribute.
- If a second index is created, the leaf vertices contain either a key or else a pointer identifying the actual record.
- If a key is kept, a second search using an index based upon that key will be required.
- If a pointer to the record is kept, that pointer must be updated if the record is moved (due to operations on the B^+-tree for the index using the key).
- It is a performance decision to choose which is best for a given situation.



Bitmap Indices

- Suppose that some survey data are given.
- Suppose further that range queries on `Sex` and `Age` are to be supported, for example:

```
SELECT * FROM Survey WHERE (SEX='F') AND (60 <= AGE) AND (AGE < 79);
```

- It may then be useful to have a *bitmap index* which allows such retrieval based upon matching of bits.
- The *bitmap* may be represented compactly as a single string.
- Standard hardware instructions for bit manipulation may then be used for rapid processing.
- The bitmap is represented as a relation, but is in fact an index on `ID` and may be implemented in a number of ways.

Survey

ID	Sex	Age	Amount	City
11111111	F	46	5321	Stockholm
22222222	F	63	5000	Gteborg
33333333	M	62	7125	Trelleborg
44444444	F	23	9100	Tillberga
55555555	M	28	1200	Tillberga
66666666	F	68	5500	Malm
77777777	F	42	5500	Simrishamn

Survey

ID	Sex	Age	Amount	City

Bitmap

ID	Sex	0-19	20-39	40-59	60-79	80-

Bitmap Indices — Additional Compactification

- To represent n conditions, only $\lceil \log(n) \rceil$ bits are required.
- This suggests the compact representation given below, using the following table.

Age Range	Encoding $A_1A_2A_3$
0-20	000
21-39	001
40-59	010
60-79	011
80-	100

Bitmap

ID	Sex	A_1	A_2	A_3
11111111	1	0	1	0
22222222	1	0	1	1
33333333	0	0	1	1
44444444	1	0	0	1
55555555	0	0	0	1
66666666	1	0	1	1
77777777	1	0	1	0

Bitmap

ID	Sex	A_1	A_2	A_3
11111111	1	0	1	0
22222222	1	0	1	1
33333333	0	0	1	1
44444444	1	0	0	1
55555555	0	0	0	1
66666666	1	0	1	1
77777777	1	0	1	0

Compact_Bitmap

ID	BitMap
11111111	1010
22222222	1011
33333333	0011
44444444	1001
55555555	0001
66666666	1011
77777777	1010

Extendible Hashing

- The goal of extendible hashing is to realize the advantage of hashing within the context of data on secondary storage:
  - Fast (constant-time) random access

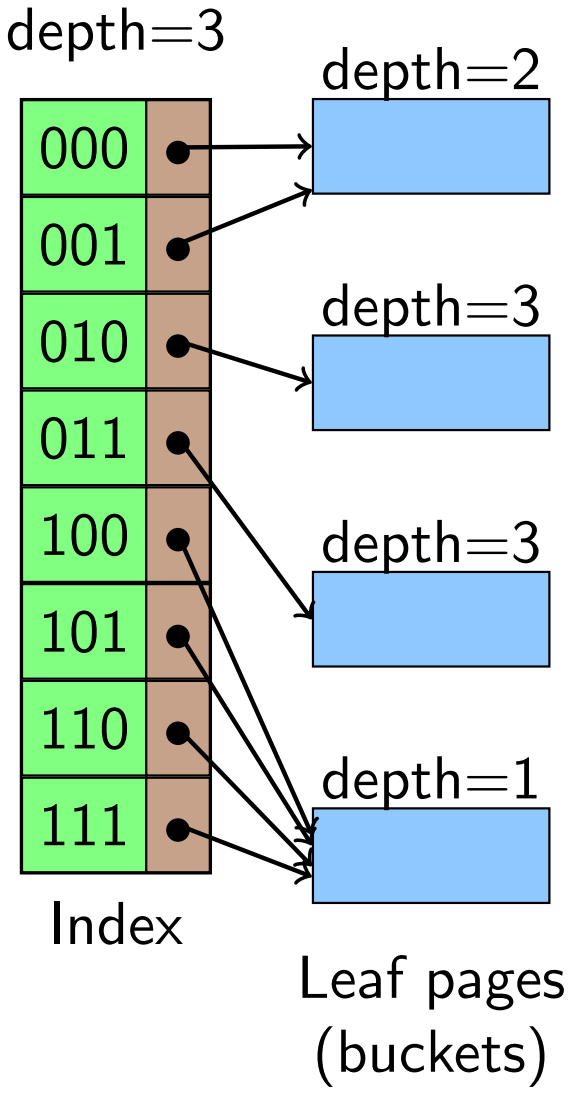
Idea: The hashing function h : keys $\rightarrow$ hash values is broken into two pieces:
(Directory address, Leaf address).

Toy example: Suppose that a two-byte hash address is used:

Directory address size: 3 bits

Hash address size: 13 bits

- Suppose that k is a key with the property that $h(k) = 1010111010110001$.
- Then, Directory address $= 101$,
  Leaf address $= 0111010110001$.
- This assumes that the first three bits are used as the directory address.

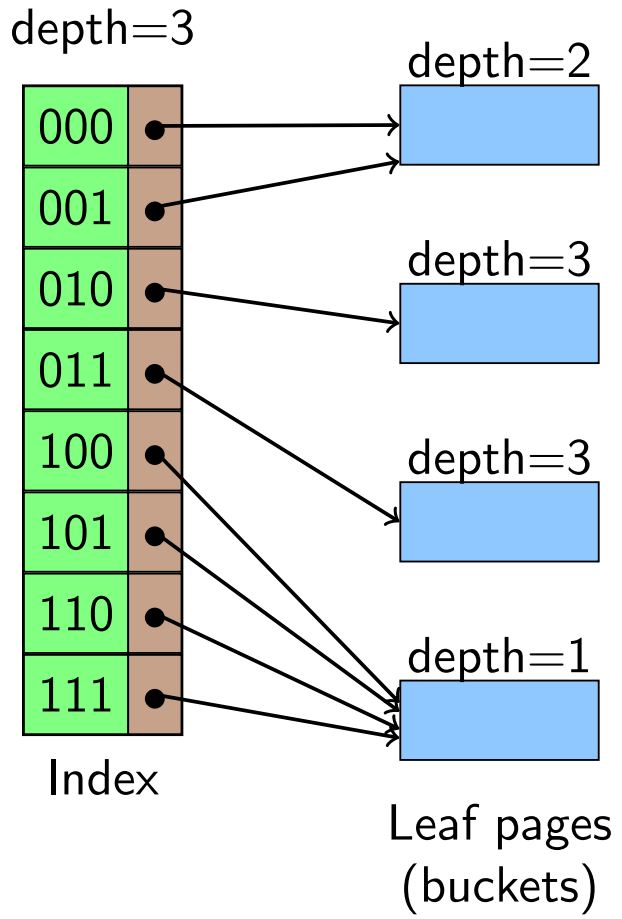


Extendible Hashing — 2

- The *depth* of an index is the number of bits of the hash value which is used as the index value.
- The *depth* of a leaf page is:

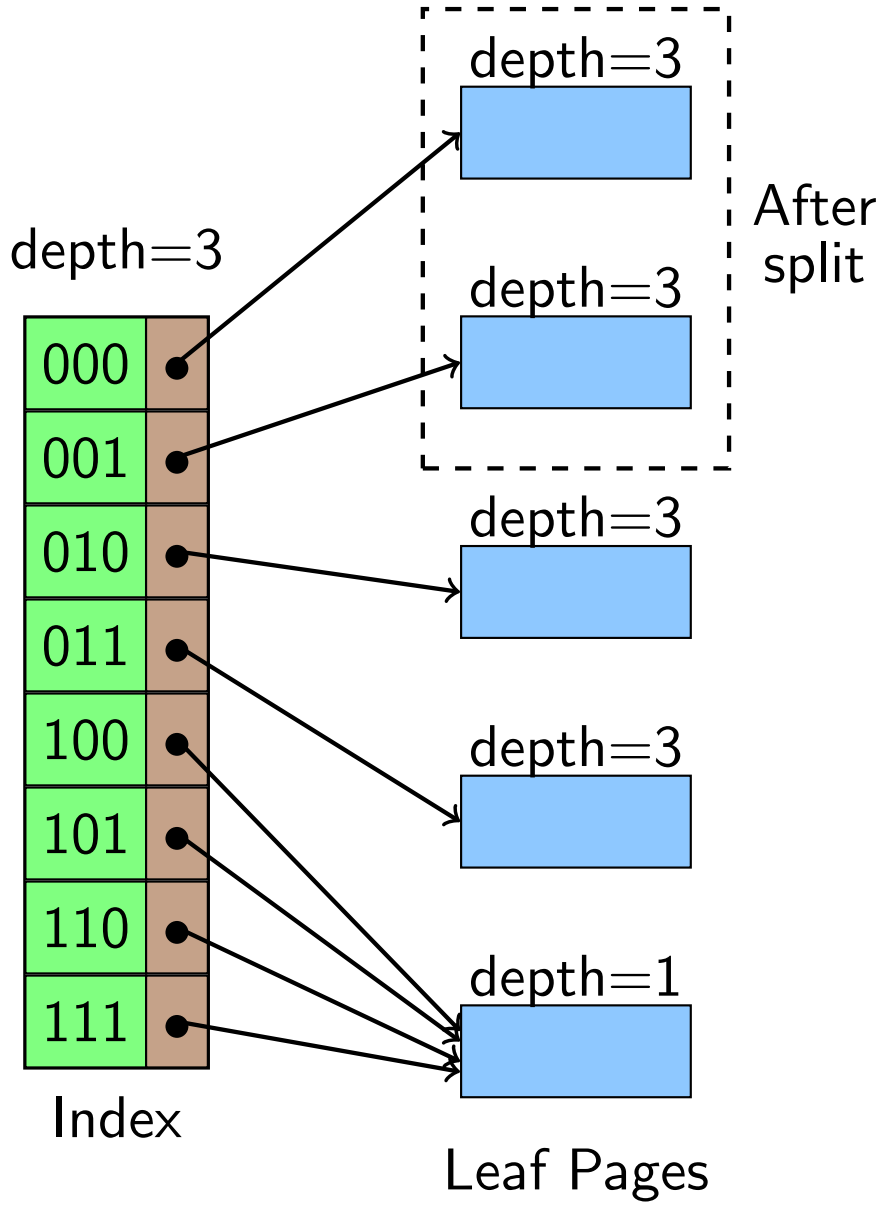
  Index depth $-$ $\log_2$(number of index entries which point to that bucket)

- The approach supports insertions quite well.
- It is less efficient at handling deletions.
- Some examples will be used to illustrate the idea.



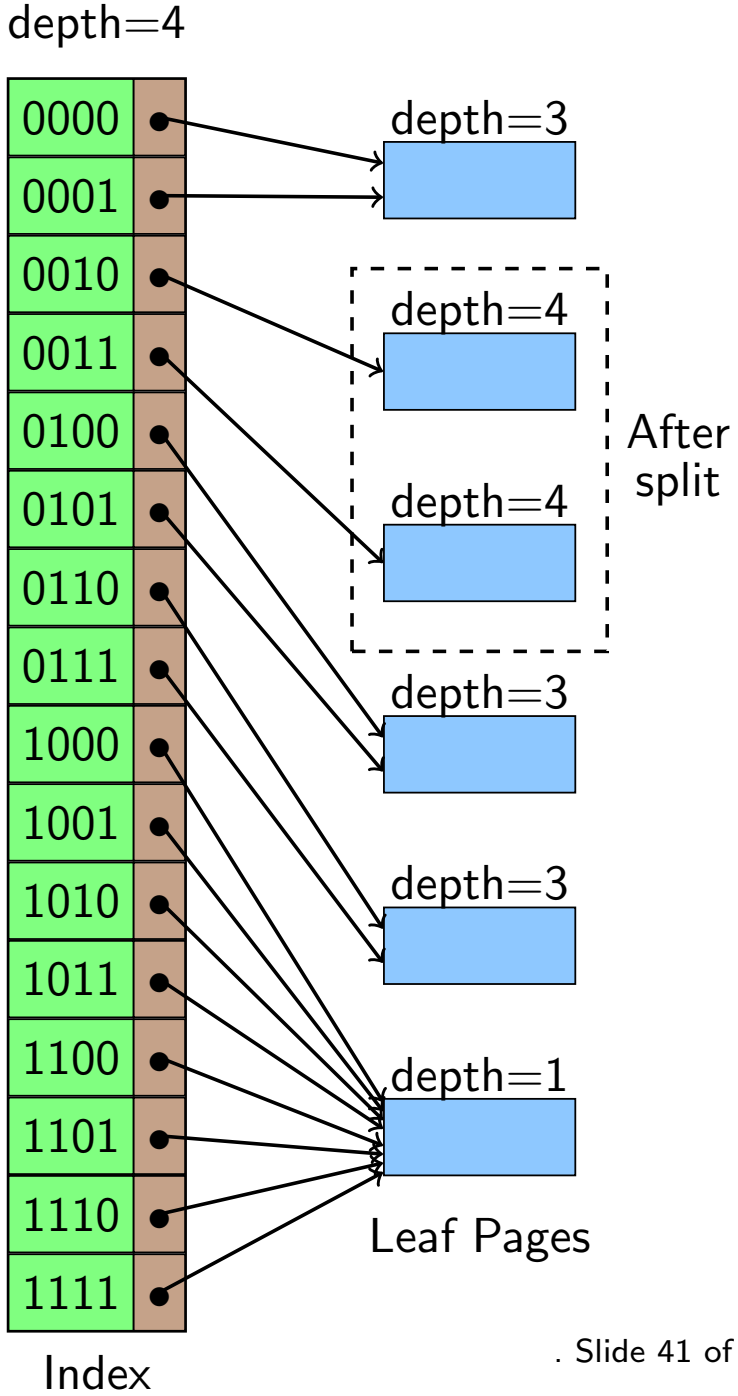
Extendible Hashing — Bucket Expansion

- Suppose that the bucket which is shared by 000 and 001 becomes full.
- To allow further insertions for keys beginning with 00, a split of this bucket is necessary.
- Notice that 000 and 001 now each have their own buckets.
- The entries of the old 000+001 bucket are divided appropriately between these two.



Extendible Hashing — Index Expansion

- Suppose that the bucket for 001 becomes full.
- To allow further insertions for keys beginning with 001, the index itself must be split.
- The depth of the index becomes four, and the number of index entries doubles.
- The entries of the old 001 bucket are are divided appropriately between the 0010 bucket and the 0011 bucket.



Remarks Regarding Extendible Hashing

- Extendible hashing works best when insertions and modifications are the dominant forms of update.
- Random-access time may be somewhat superior to that for B^+-trees, particularly when memory is limited.
  - The index for extendible hashing may be much smaller than the index for a corresponding B^+-tree.
  - No searching is required; just computation of a key-to-address transformation and an array access.
  - Relative advantages diminish as memory size increases.
- With a typical hashing strategy: Sequential processing becomes very slow.

Indices in PostgreSQL

- PostgreSQL supports *extensible indices*.
  - These indices may be created and modified dynamically, after the initial schema is declared and built.

Ordinary index types:

B+-tree index: The default and the most common type.
- The entire structure (index+data) is essentially a B^+-tree.
- A B+-tree index on the primary key of each relation is created automatically.

Hashed index: Not widely used; must be rebuilt after system crash.
- Not clear whether extendible hashing is used; no mention.

Generalized index types: Apply to text structures

Generalized inverted index (GIN) : Lossless, faster access, better for static data.

Generalized search tree (GiST): Lossy, faster to update, better for dynamic data.

Declaring Indices in PostgreSQL

- The following directive creates a B+-tree index named `sn` on the attribute `dept_name` of the `student` relation.

```
CREATE INDEX sn ON student (name);
```

- The presence of the index may be observed using the `\d` directive.

```
university=# \d student
             Table "public.student"
  Column   |          Type           | Modifiers
-----------+-------------------------+-----------
 id        | character varying(5)    | not null
 name      | character varying(20)   | not null
 dept_name | character varying(20)   |
 tot_cred  | numeric(3,0)            |
Indexes:
    "student_pkey" PRIMARY KEY, btree (id)
    "sn" btree (name)
...
```

- The index may be dropped as follows.

```
DROP INDEX sn;
```

- Indices may be created without specifying a name, in which case a system-generated name is assigned.

```
CREATE INDEX ON student (name);
```

Partial and Transformed Indices in PostgreSQL

- Indices may be partial, specified by a condition.

```
CREATE INDEX sdncsee ON student (dept_name) WHERE
      (dept_name = 'Comp. Sci.' OR dept_name = 'Elec. Eng.');

CREATE INDEX scr50 ON student (tot_cred) WHERE (tot_cred>50);
```

- The condition must be static and not depend upon the state of the database.
  - In particular, a subquery is not allowed.
- A transformation of the index value, via a function, is also permitted.
- The following creates an index on the lower-case equivalent of the student names.

```
CREATE INDEX lower_name ON student (LOWER(name));
```

- This helps with the following query.

```
university=# SELECT * FROM student WHERE LOWER(name) LIKE 's%';
  id   |  name   | dept_name  | tot_cred
-------+---------+------------+----------
 12345 | Shankar | Comp. Sci. |       32
 55739 | Sanchez | Music      |       38
 70557 | Snow    | Physics    |        0
(3 rows)
```

Multi-Attribute Indices in PostgreSQL

- Indices may be over several attributes.

```
CREATE INDEX sndn ON student (name,dept_name);
```

- The index is created by combining the attributes in a left-to-right fashion.
- This index might thus even be useful to find tuples matching a value for attribute `name`, but is not likely to be useful in finding matches to `dept_name` only.

Clustering and Reclustering Tables in PostgreSQL

- By default, the records of a relation are clustered in the order defined by the primary key.
- However, this may be changed by using the `CLUSTER` directive on an existing index.

```
CREATE INDEX sn ON student (name);
CLUSTER student USING sn;
```

- Remember that a table can be clustered by only one index.
- Initially, a table is clustered perfectly; the records are in the order specified by the index.
- As updates occur, particularly insertions, this clustering becomes compromised.
- It is possible to recluster an index; the same directive is used, but the index need not be given.

```
CLUSTER student;
```

- This restores physical contiguity but does not change the index (much) itself.
- Without an argument, all existing indices are reclustered.

```
CLUSTER;
```

Reindexing Indices and Tables in PostgreSQL

- Reclustering rebuilds the leaves of a B^+-tree index, but does not make substantial changes to the index structure itself.
- Over time, the index can become inefficient, with many pages which are only minimally populated.
- Also, an index can occasionally become corrupted.
- The `REINDEX` command rebuilds the index itself.

```
REINDEX INDEX sn;
```

- It is also possible to reindex all indices in a table, all tables in a database, or even an entire system, by using the appropriate keyword.
- The following command reindexes all indices in the `student` relation.

```
REINDEX TABLE student;
```

- All of these directives lock (for both read and write) all tables involved.

The Fill Factor of Indices in PostgreSQL

- The *fill factor* of an index determines how much free space is left in the vertices of the index itself.
  - The default at index creation is 90%.
  - $10\% \leq$ fill factor $\leq 100\%$.
  - Thus, it need not be $\geq 50\%$, as in a true B+-tree.
    - However, such a low fill factor would be unusual.
  - It may be set to a non-default value when the index is created or via an `ALTER INDEX` command.
  - In the latter case, `REINDEX` must be used afterwards.
  - A higher fill factor results in a shallower tree, with better read performance, but may require deep splits to support insertions.

Hash and Other Index Types in PostgreSQL

- Hash indices are created with the keyword `HASH`.

```
CREATE INDEX sn ON student USING HASH (name);
```

- Hash indices typically do not offer strong advantages over B+-tree indices, and so are not widely used.
- *GIN* and *GiST* indices similarly created using the keywords `gin` and `gist`, respectively, but they are restricted to attributes of type *tsvector* and *tsquery*.

Choosing and Evaluating Indices in PostgreSQL

Question: How does one decide which types of indices to try, and how does one evaluate their performance.

Answer: The short answer is that *DBMS tuning* requires great skill, and those who are good at it command high salaries.

- Nevertheless, it is possible for anyone to experiment.
- A limitation is that quite large databases are often required in order to see the benefits and drawbacks of indices.
- For smaller databases which fit in main memory, sequential scans, sometimes combined with temporary hash tables, are so fast that indices provide little advantage, so the query optimizer does not use them.
- For serious experiments, it is often necessary to work with databases which are big enough that the relations necessary to evaluate a query do not fit in main memory.
- This involves sizes which are measured in gigabytes, at least.